

API TESTING MASTERCLASS

AI Prompt Library for API Testing

Ready-to-use prompts to accelerate API testing — used responsibly.

Using AI Responsibly for API Testing

AI is a fast assistant for generating test ideas, assertions, data and documentation — but it is an assistant, not an oracle. It does not know your system and it will confidently invent endpoints, fields and status codes that do not exist. Everything it produces is a draft you must verify against the real API and its documentation before you rely on it.

- Never paste real secrets, API tokens, passwords, customer data or anything confidential into an AI tool. Redact and use placeholders (e.g. {{token}}, user@example.com).
- Treat every generated endpoint, field name and status code as unverified until you have checked it against the actual API and docs.
- Use AI to widen your thinking, not to replace it — you still own the sign-off decision.
- Prefer sanitised, representative examples over production payloads when you need to give the model context.

WATCH OUT — HALLUCINATED ENDPOINTS & FIELDS

AI regularly fabricates endpoints, query parameters, response fields and status codes that sound plausible but do not exist in your API. Never add a test for something just because the model suggested it — confirm it against the real contract or documentation first, or you will ship tests that assert fiction.

Endpoint Analysis

Prompt: "Here is an API endpoint description (or OpenAPI snippet): [paste sanitised spec]. Explain in plain English what this endpoint does, its inputs (path, query, body), its expected responses and status codes, and any authentication it requires. List anything that is ambiguous or missing from the description."

- How to use: paste a sanitised spec or the endpoint docs, then read the explanation to confirm your own understanding.
- What to check: that the inputs, responses and auth it describes actually match the real endpoint — the "ambiguous or missing" list is the most valuable part.

Generating Test Ideas

Prompt: "Act as a senior QA engineer. For this endpoint [paste sanitised description], list test scenarios grouped into: positive/happy path, negative and error handling, boundary and edge cases, authentication and authorisation, and data validation. For each, state the scenario and the expected result."

- How to use: use the output as a checklist to expand your coverage, then prune anything that does not apply.

- What to check: that expected results are realistic for your API, and add domain-specific cases the model cannot know about.

Writing Assertions

Prompt: "Here is a sample JSON response: [paste sanitised response]. Write Postman test assertions (using `pm.test` and `pm.expect`) that verify the status code, key fields exist and have the correct types, and a couple of business rules I describe: [describe rules]. Keep assertions specific and independent."

- How to use: paste a real (sanitised) response so assertions match the actual shape, then drop them into the Tests tab.
- What to check: that field names and types match the real payload exactly, and that no assertion depends on volatile data like timestamps or generated IDs.

Generating Test Data

Prompt: "Generate a CSV of 10 test data rows for this request. Columns: [list fields]. Include valid values, boundary values (min, max, just over), and invalid values (wrong type, empty, too long). Use fake, non-personal data only, and add a column describing the expected outcome for each row."

- How to use: feed the CSV into a Newman data-driven run (`-d data.csv`) to exercise many cases at once.
- What to check: that boundaries reflect your real limits, the data is entirely synthetic, and the expected-outcome column is correct.

Writing API Docs

Prompt: "Turn this endpoint and sample request/response [paste sanitised examples] into clear API documentation: purpose, method and path, request parameters and body, an example request, an example response, and the possible status codes with their meaning. Write for a developer who has never used this API."

- How to use: use the draft as a starting point for a README or handover note, then edit for house style.
- What to check: every parameter, field and status code against the live API — accurate docs matter more than polished prose.

Reviewing a Response

Prompt: "Here is a request and the response I received [paste sanitised request + response]. Does the response look correct and consistent for this request? Point out anything suspicious: wrong status code, missing or unexpected fields, inconsistent types, values that contradict the request, or signs of an error being returned as a success."

- How to use: use it as a second pair of eyes when a response "looks fine" but you want a sanity check.
- What to check: confirm any issue it raises against the spec — and remember it cannot know your business rules unless you tell it.

INSIDE STLC PRO TIP

Keep your best prompts in a shared team file and refine them as you learn what your API returns. A tuned prompt that includes a real (sanitised) response and your specific business rules produces far more useful, less hallucinated output than a generic one-liner.